

ICLR 2024 | Teaching Mixture Models to Subtract: Square It, and It Stays Both Compact and Valid

Researchers from Edinburgh, Aalto, and collaborators introduce NPC²: squaring turns subtractive mixtures into exactly-normalizable probabilistic circuits that are provably, exponentially more compact than additive ones

A mixture model is one of the oldest and most dependable tools in probabilistic modeling: take a handful of simple distributions, say a few Gaussians, weight them by non-negative coefficients, and add them up to form something more complex. This additive recipe is simple and safe, but it carries a hidden cost. When the target distribution has holes in it, or sharp structure, an additive mixture has to pile up a great many components just to carve those holes out one piece at a time. That is a wasteful way to spend parameters.

The natural fix is to let some components subtract probability mass instead of only adding it. A mixture that can cancel mass can often capture the same complicated shape with far fewer components. The catch is that once you allow negative weights, the function can dip below zero and stop being a valid probability distribution, and learning it becomes genuinely hard. That is the problem this ICLR 2024 paper sets out to solve.

The answer the authors give is disarmingly clean: square the whole mixture. Squaring forces the output to be non-negative no matter what the weights are, so the coefficients are free to go negative, and the squared model can still be normalized exactly. The authors express this construction uniformly in the language of tensorized probabilistic circuits, arriving at squared non-monotonic probabilistic circuits (NPC²s), and prove that these can be exponentially more compact than any additive, monotonic circuit.

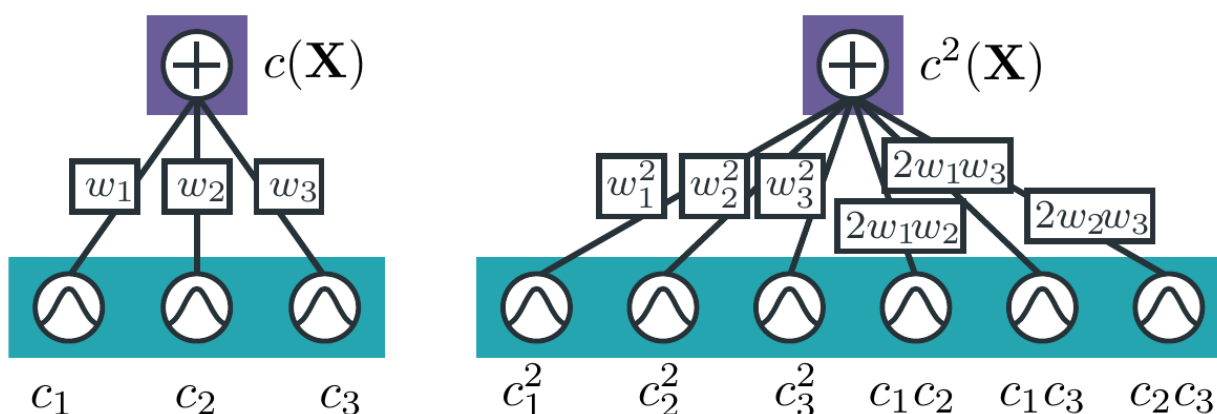


Figure 1. Squaring a shallow mixture $c(\mathbf{X})$ with $K=3$ components to obtain $c^2(\mathbf{X})$. Left: an ordinary additive mixture, three components with weights w_1, w_2, w_3 . Right: the equivalent circuit after squaring. Expanding the square yields all pairwise products of components ($c_1^2, c_2^2, c_3^2, c_1c_2, c_1c_3, c_2c_3$) with weights $w_i \cdot w_j$. The number of components grows from 3 to 6, but every one of those weights is shared from the original 3 parameters.

Why additive mixtures are a bad deal

A classical mixture requires its component weights to be non-negative and to normalize, which makes it a convex combination of simple distributions. Every term in a convex combination can only push probability mass upward; terms never cancel each other. So if you want a hole in the density, a region where the probability should be very low or near zero, an additive model can only approximate it by crowding components around the hole and relying on their tails to suppress one another. It works, but the parameter efficiency is poor.

Letting components cancel each other out addresses the problem at its root: a subtractive mixture can describe distributions with holes and sharp features using far fewer components than an additive one. But there is no free lunch. Negative weights can drive the overall function below zero at some points, and a negative probability density is meaningless. Energy-based models sidestep non-negativity by exponentiating, but they pay for it by losing tractable normalization. Meanwhile, several disconnected communities, from signal processing to kernel methods to quantum mechanics, each reinvented some flavor of a squaring trick to enforce non-negativity, without a common, tractable framework tying them together.

The core trick: square the mixture

Start with the simplest, shallow case. Take a K -component mixture $c(X) = \sum_i w_i c_i(X)$ whose weights are unconstrained, positive or negative, and square it: $c^2(X) = (\sum_i w_i c_i(X))^2 = \sum_i \sum_j w_i w_j c_i(X) c_j(X)$, with $w_i \in \mathbb{R}$. After squaring the output is non-negative regardless of the signs of the weights, so validity comes for free.

Expanding the square turns the K -component mixture into a product-of-experts form over all pairs of components, exactly the right side of Figure 1. The key point is that normalization is still computable: the partition function $Z = \int c^2(x) dx = \sum_i \sum_j w_i w_j \int c_i(x) c_j(x) dx$ can be evaluated in closed form whenever each pairwise product integrates in closed form, which holds for many common families. The log-likelihood then reads $\sum_x \log(c^2(x)/Z) = -|D| \log Z + 2 \sum_x \log|c(x)|$, so the model trains by gradient-based maximum likelihood like anything else.

Going deep: square a tensorized circuit layer by layer

Shallow squaring is just the warm-up; the real expressiveness comes from depth. The authors lift the construction to deep, structured-decomposable tensorized circuits, which are built by stacking input layers, product layers (Hadamard products), and sum layers (each parameterized by a matrix). Squaring the whole circuit reduces to squaring each layer: working bottom-up, you recursively square every layer, and the result is still a valid deep circuit, NPC², that remains tractable for exact marginalization and inference.

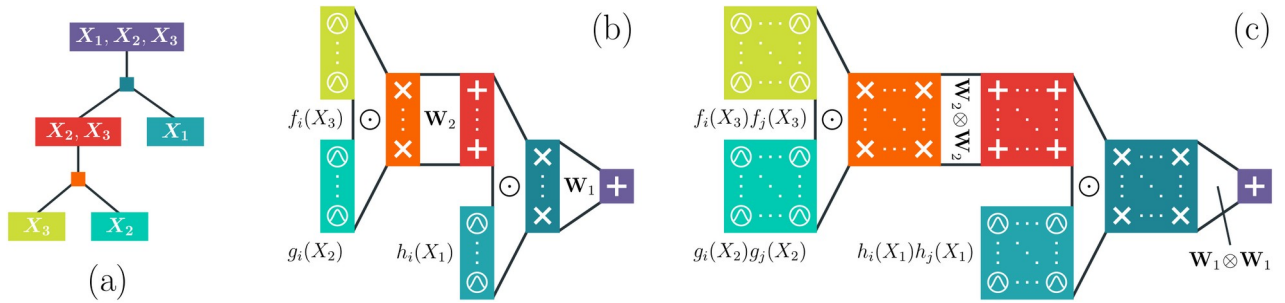


Figure 2. Squaring a tensorized structured-decomposable circuit reduces to squaring its layers. (a) A region graph over variables X_1, X_2, X_3 ; (b) the circuit defined from it, with colored boxes for input, product ($\odot$), and sum (matrix W) layers; (c) the result of recursively squaring each layer. Squared layers hold a quadratic number of units, yet each layer still outputs a vector, so the model stays efficient.

There is an obvious worry here: after squaring, the unit count becomes quadratic, so won't training grind to a halt? It won't. Although a squared layer contains a quadratic number of units internally, each layer still emits a vector rather than a matrix, and the partition function Z is computed just once per batch. As a result, NPC²s train stably with gradient-based maximum likelihood, much like a standard neural network.

Visible evidence: carving holes in 2D densities

However clean the theory, it has to work on data. The authors start with two-dimensional synthetic densities, the most direct test possible, because a 2D distribution can be plotted and judged by eye. Three models are in play: a plain monotonic circuit (MPC, additive only), a squared monotonic circuit (MPC², which squares but keeps all weights non-negative), and a squared non-monotonic circuit (NPC², which both squares and allows negative weights).

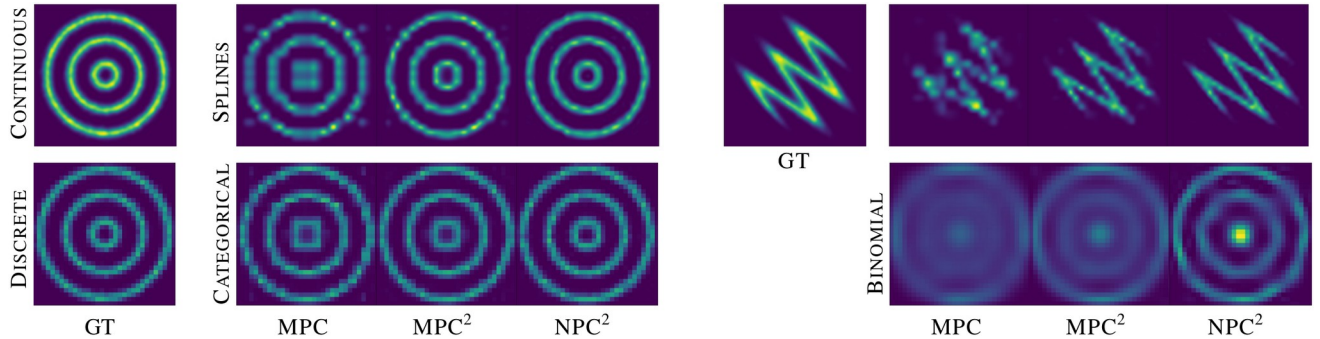


Figure 3. Densities estimated by MPC, MPC², and NPC² on 2D continuous (top) and discrete (bottom) data.

The concentric-ring targets on the left have clear holes between the rings. Continuous inputs use splines; discrete inputs use categoricals, embeddings (for NPC²s), or Binomials. NPC² renders the gaps between the rings most cleanly, and the advantage is sharpest with parameter-efficient input layers.

The comparison is telling. Squaring alone (MPC²) already improves on the purely additive MPC, but the model that actually carves the holes between the rings cleanly is the one with negative weights, NPC². In other words, the extra expressiveness does not come from squaring per se; it comes from the negative parameters, and squaring is simply the mechanism that lets those negative parameters exist legally. The same pattern holds on the discrete data in the bottom row.

On standard benchmarks: same size, higher likelihood

Next comes the harder quantitative test. On five widely used UCI multivariate density-estimation datasets (Power, Gas, Hepmass, MiniBooNE, BSDS300), the authors compare NPC²s against a spread of strong baselines: normalizing flows (RealNVP, MADE, MAF, NSF), a full-covariance Gaussian, monotonic probabilistic circuits (EiNet-LRS), and tensor-train density estimators (TTDE).

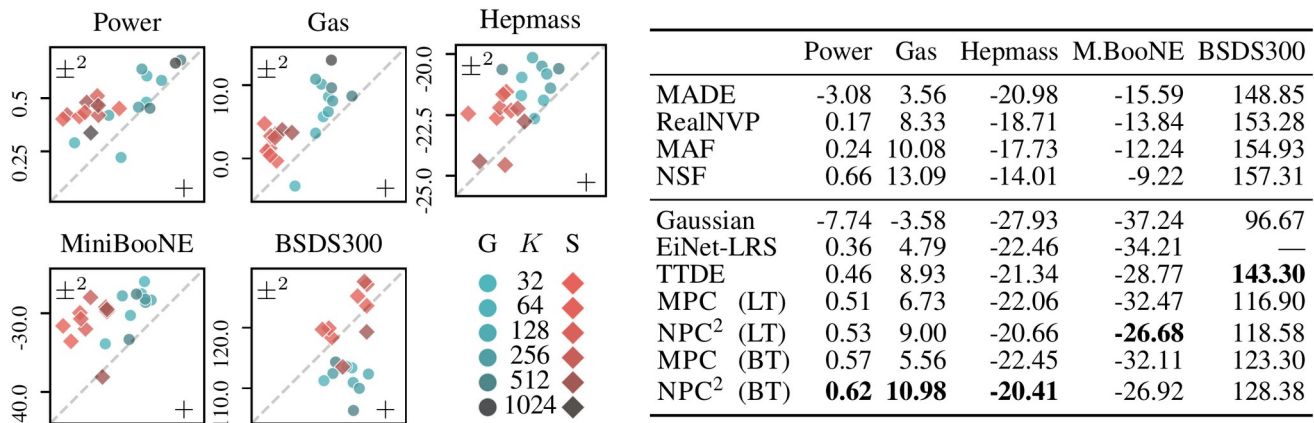


Figure 4. Left: on the five UCI datasets, NPC²s ($\pm^2$) are paired point-for-point against monotonic circuits (+) at matched model size; darker points use larger per-layer width K , blue marks Gaussian input layers and red marks splines. Points above the diagonal are NPC²s reaching higher log-likelihood at parity of size. Right: best average test log-likelihoods for the authors' models against intractable (top) and tractable (bottom) baselines.

Most points in the scatter plots sit above the diagonal, meaning that at a matched parameter budget, NPC²s reach higher log-likelihood than monotonic circuits. The table on the right puts numbers to it: among tractable models, NPC²s attain the best average test log-likelihood on several datasets. The compact table below lists a few tractable models for quick reference.

Table 1. Best average test log-likelihoods on the five UCI datasets for tractable models (higher is better; EiNet-LRS has no BSDS300 result, shown as —). NPC² (BT) is best in its class on Power, Gas, and Hepmass.

Model	Power	Gas	Hepmass	MiniBooNE	BSDS300
Gaussian	-7.74	-3.58	-27.93	-37.24	96.67
EiNet-LRS	0.36	4.79	-22.46	-34.21	—
TTDE	0.46	8.93	-21.34	-28.77	143.30
MPC (BT)	0.57	5.56	-22.45	-32.11	123.30
NPC ² (BT)	0.62	10.98	-20.41	-26.92	128.38

It is worth being honest about the boundary: the advantage is not uniform across datasets. On the image-texture dataset BSDS300, for instance, TTDE leads at 143.30, and on some discrete tasks whose inputs are already expressive, NPC²'s margin narrows. The authors' ablations add nuance: binary-tree region graphs generally beat linear-tree ones, splines help most on continuous data, and embeddings or Binomials matter more than categorical on discrete data.

Distilling GPT2: approximating an intractable model

The last experiment is the most intriguing: can a tractable circuit approximate a large model that is itself intractable? The authors sample sentences from GPT2, train NPC²s and monotonic circuits on that data, and ask which better approximates GPT2's distribution. For reference, GPT2's own log-likelihood on the same training data is about -52.

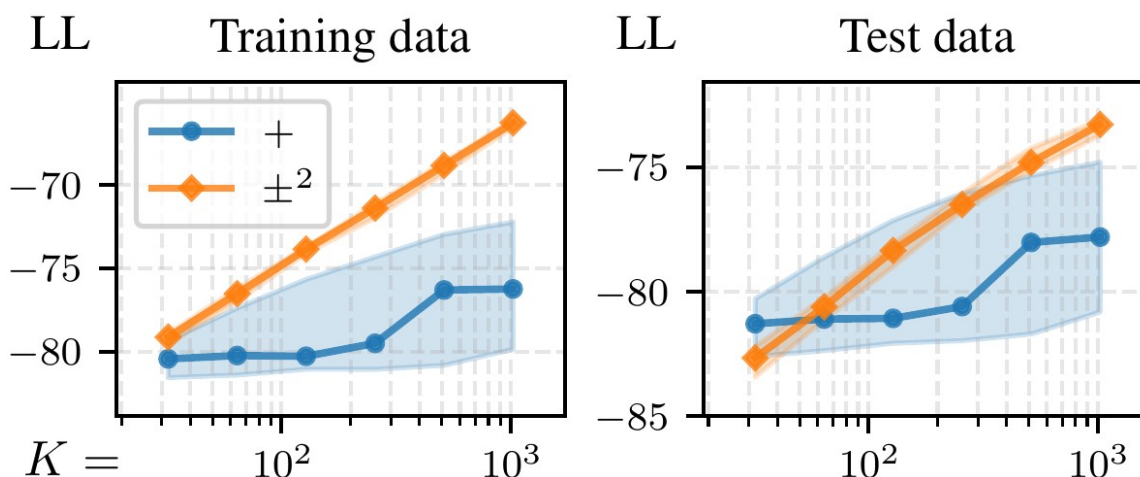


Figure 5. On data sampled from GPT2, log-likelihoods of NPC²s ($\pm^2$, orange) and monotonic circuits (+, blue) as the per-layer width K grows. Lines show the median; the shaded band covers 90% of runs. As K increases, NPC²s keep climbing toward GPT2's reference log-likelihood of about -52, while monotonic circuits plateau early. The gap on test data is significant for most values of K .

The conclusion is clear: as the model grows, NPC²s scale distinctly better, with log-likelihood climbing steadily toward GPT2's own level, whereas monotonic circuits stop improving sooner. At matched size, then, squared non-monotonic circuits approximate this otherwise-intractable language model more faithfully.

Takeaway: making subtraction a first-class citizen

The upshot fits in a sentence: squaring turns subtractive mixtures into valid, tractable probabilistic models that need exponentially fewer components than additive mixtures to reach the same expressiveness. Squaring a K -component mixture encodes on the order of $(K+1 \text{ choose } 2)$ pairwise components while reusing the same K parameters, which is exactly where the compactness comes from, and the authors prove that this compactness gap over monotonic circuits is exponential.

Just as notably, this squaring-centered, property-driven view of circuits unifies negative-parameter models that had been scattered across fields: square-root-of-density models, positive semi-definite kernel models, and Born machines from quantum physics all fall under one framework. For practitioners, it promotes subtraction from an ad hoc trick to a first-class tool you can reach for when building compact, expressive, and tractable probabilistic models.